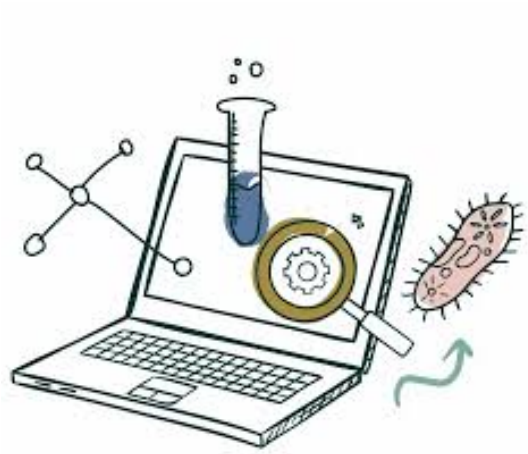
Système de Surveillance des Données de Pression Artérielle patient avec Kafka, Elasticsearch et Kibana

M1 Projet Data Science 2025/2026



Auteurs:
BECHIR YOUSSOUF Awatif
MAHFOOZ Nabiha

Université Sorbonne Paris Nord

Table des matières

1 : Introduction

- Contexte médical
- Présentation FHIR
- Objectifs techniques globaux

2 : Architecture Générale du Système

- Vue d'ensemble du pipeline
- Rôle des composants

3 : Implémentation Technique

3 a : Génération des messages FHIR
Structure JSON
Faker
Modélisation et Observation (FHIR)

3 b : Mise en place du flux Kafka
Producer
Topic
Consumer
Streaming temps réel

3 c Détection des anomalies
Explication des seuils médicau
Logique de classification
Types d'anomalies
Justification médicale

3 d Traitement et stockage des données
Séparation normal / anormal
Indexation Elasticsearch
Sauvegarde JSON locale
Justification du stockage différencié

4 Visualisation et Analyse des Résultats
Mapping
Connexion Kibana
Structure des dashboards
Interprétation des dashboards
Analyse des anomalies détectées
Tendances observées

Cas critiques

5 Discussion

Limites des seuils
Limites du modèle
Améliorations possibles
Sécurité des données médicales

Conclusion

1 Introduction :

La pression artérielle est un indicateur central dans le domaine hospitalier, car des valeurs anormales peuvent révéler des pathologies graves comme l'hypertension ou l'hypotension. Avec la généralisation des objets connectés de santé, une surveillance automatisée et en temps réel devient cruciale pour améliorer la réactivité des soins.

Ce défi s'inscrit dans les problématiques du Big Data, marqué par la variété et le volume des données produites par les dispositifs médicaux, souvent structurées au format JSON via le standard FHIR. Une architecture robuste et évolutive, assurant l'interopérabilité grâce au standard FHIR, permet d'intégrer efficacement ces observations dans des systèmes diversifiés.

Pour répondre à ces contraintes, le projet s'appuie sur Apache Kafka qui assure l'ajout continue des messages FHIR et leur découplage entre producteurs et consommateurs. Les données critiques sont indexées dans Elasticsearch pour permettre des requêtes rapides sur les anomalies, et visualisées via Kibana sous forme de tableaux de bord facilitant ainsi l'analyse et la décision médicale.

L'objectif est de concevoir une architecture Big Data complète, de la génération de messages FHIR synthétiques jusqu'à la visualisation dynamique, illustrant l'application concrète des principes de streaming en temps réel à la surveillance médicale numérique.

2 Architecture général du système :

Vue d'ensemble du pipeline de données

L'architecture repose sur un modèle de traitement en flux temps réel, où les mesures de pression artérielle sont analysées dès leur réception afin de permettre une détection rapide des anomalies.

Les données structurées selon le standard médical FHIR passent par quatre étapes successives:

1. Génération simulée : un Producer Python génère des ressources FHIR contenant les valeurs systoliques et diastoliques de patients fictifs à l'aide de la bibliothèque fhir.resources.

2. Transit via Kafka : les messages sont publiés dans le topic blood_pressure et mis à disposition du Consumer en temps réel.

3. Analyse et détection : un Consumer Python consomme chaque message, extrait les valeurs de pression et applique les seuils médicaux définis pour identifier les anomalies (hypertension/hypotension).

4. Stockage différencié : les cas normaux sont archivés localement au format JSON, tandis que les cas anormaux sont indexés dans Elasticsearch et visualisés en temps réel dans Kibana.

Rôle des composants du système

Le générateur FHIR (Python) sert de source de données, produisant un format médical JSON standardisé proche de ce qui enverrait de vrais dispositifs. Chaque message contient une observation avec un identifiant patient, des valeurs systolique/diastolique et un horodatage. Ce format est compatible avec Kafka et Elasticsearch qui gèrent les documents.

Apache Kafka est au centre de la circulation des données, agissant comme système de messages distribué : le producteur envoie les observations, et Kafka les stocke dans un topic.

3 Implémentation technique :

a: Génération des messages FHIR

La première étape du pipeline consiste à produire un flux continu d'observations de pression artérielle simulées en l'absence de dispositifs médicaux réels, un module Python joue le rôle de source de données, générant des mesures synthétiques représentant des observations cliniques patient.
L'objectif est de simuler un environnement réaliste dans lequel des dispositifs médicaux enverraient régulièrement des mesures vers une infrastructure centrale.
Le format JSON simplifié a été retenu pour les échanges entre le Producer et le module d'analyse afin de faciliter le traitement côté Consumer notamment pour l'implémentation des règles de détection d'anomalies.

Le message généré contient les champs suivants :

- patient_id : identifiant court du patient
- name: nom complet généré aléatoirement
- systolic : pression artérielle systolique (mmHg)
- diastolic : pression artérielle diastolique (mmHg)
- timestamp : date et heure de la mesure au format ISO 8601

Les valeurs de pression artérielle sont simulées dans des plages larges :

- systolique : 70 – 200 mmHg
- diastolique : 40 – 130 mmHg

Ces intervalles dépassent les seuils médicaux normaux (systolique : 90–140 mmHg, diastolique : 60–90 mmHg) afin de garantir la présence régulière de cas anormaux dans le

flux généré et pour tester le module de détection d'anomalies développé dans la suite du projet.

La génération repose sur les bibliothèques Python faker et random. La bibliothèque faker permet de produire des identités et des horodatages réalistes, tandis que random assure la variabilité des valeurs physiologiques simulées.

b: Mise en place du flux Kafka

Après la génération des observations de pression artérielle, la deuxième étape du pipeline consiste à organiser leur transport en continu vers le module d'analyse. Pour cela, nous utilisons Apache Kafka pour gérer des flux d'événements en temps réel.

Kafka joue ici le rôle d'intermédiaire entre deux composants:
- le Producer, qui envoie les observations,
- le Consumer, qui les récupère pour traitement.

Dans notre projet, les messages transmis contiennent à la fois des champs simples (patient_id, systolic, diastolic, timestamp) utilisés directement par le module d'analyse, ainsi qu'une ressource FHIR complète de type Observation construite avec la bibliothèque fhir.resources.

Producer

Le Producer est responsable de l'envoi des observations vers Kafka.

Son fonctionnement repose sur quatre étapes :

1. Génération d'une observation
2. Conversion du dictionnaire Python en JSON
3. encodage en UTF-8 (Kafka transmet des bytes)
4. Publication dans le topic blood_pressure

Dans notre implémentation, la génération de l'observation repose sur la bibliothèque fhir.resources afin de modéliser une véritable ressource FHIR Observation conforme au standard HL7. Les codes LOINC utilisés sont 85354-9 (Blood pressure panel), 8480-6 (Systolic blood pressure) et 8462-4 (Diastolic blood pressure), ce qui assure une représentation médicalement standardisée des données.

Le Producer utilise la bibliothèque confluent_kafka, cohérente avec les exemples du cours. L'adresse du broker et le nom du topic sont configurables via des variables

L'utilisation de variables d'environnement (KAFKA_BOOTSTRAP_SeRVeRS, KAFKA_TOPIC, INTeRVAL_SeCONDS) permet de rendre l'application facilement déployable dans différents environnements (local, Docker, cloud) sans modification du code source.

Chaque message envoyé est confirmé grâce à un callback (delivery_report), permettant de vérifier que Kafka a bien enregistré l'événement dans le topic. L'affichage des offsets dans le terminal confirme le bon fonctionnement du streaming en temps réel

c. Détection des anomalies

La fonction détecter anomalies (systolic, diastolic) reçoit les deux valeurs de pression d'un patient. Elle commence par créer une liste vide appelée anomalies. Cette liste servira à stocker les anomalies détectées.
Ensuite, elle compare la valeur systolique aux seuils définis. Si la valeur est supérieure à 140 mmHg, l'anomalie "hypertension_systolique" est ajoutée à la liste. Si elle est inférieure à 90 mmHg, l'anomalie "hypotension_systolique" est ajoutée.

La même logique est appliquée à la pression diastolique.
À la fin de la fonction, la liste des anomalies est renvoyée :

- Si la liste est vide, le patient est considéré comme normal.
- Si la liste contient au moins un élément, le patient est considéré comme anormal.

Cette logique est simple mais efficace car elle permet de détecter plusieurs anomalies simultanément chez un même patient.

Types d'anomalies détectées

Le système peut détecter quatre types d'anomalies :

hypertension_systolique
hypotension_systolique
hypertension_diastolique
hypotension_diastolique

Un patient peut présenter une seule anomalie ou plusieurs en même temps.

Justification médicale

Les seuils choisis correspondent aux recommandations médicales couramment utilisées pour diagnostiquer l'hypertension et l'hypotension chez l'adulte.

L'hypertension est un facteur de risque majeur pour les maladies cardiovasculaires, les accidents vasculaires cérébraux et les insuffisances cardiaques. Une pression trop élevée augmente la charge exercée sur les artères et le cœur.

L'hypotension, bien que souvent moins grave, peut provoquer des malaises, des vertiges ou une perte de connaissance si la pression est trop basse.

Ainsi, même si les données sont simulées, la logique de détection repose sur des bases médicales cohérentes.

3.d Traitement et stockage des données

Séparation normal / anormal

Le module Stockage_traitement_données.py reçoit les données du consommateur Kafka via la fonction traiter_donnees().
Cette fonction appelle detecter_anomalies() afin de déterminer si le patient est normal ou anormal. Si aucune anomalie n'est détectée, les données sont considérées comme normales. Si au moins une anomalie est détectée, les données sont considérées comme anormales. Cette séparation permet de distinguer les cas standards des cas nécessitant une attention particulière.

Indexation Elasticsearch

Lorsque le patient est anormal, ses données sont envoyées dans Elasticsearch dans l'index blood_pressure_anomalies.

Un document est créé avec les champs suivants :-
- patient_id
- name
- systolic_pressure
-diastolic_pressure
- anomaly_type
- timestamp

Elasticsearch permet ensuite d'exploiter ces données dans Kibana pour créer des visualisations en temps réel.
L'indexation permet une recherche rapide, des agrégations statistiques et une visualisation dynamique des anomalies.

Sauvegarde JSON locale

Les patients normaux sont enregistrés localement dans le fichier patients_normaux.json.

Chaque patient est écrit sur une ligne différente au format JSON. Cette méthode est simple et permet d'archiver l'ensemble des données sans surcharger Elasticsearch avec des données normales.

Justification du stockage différencié

Le choix de séparer le stockage repose sur une logique d'optimisation. Les données normales représentent la majorité du flux. Les stocker dans Elasticsearch serait inutilement coûteux en termes de performance et de stockage.
En revanche, les anomalies sont peu nombreuses mais importantes. Elles doivent être facilement consultables, filtrables et analysables. Elasticsearch est donc parfaitement adapté

pour cela. Cette stratégie améliore les performances globales du système tout en conservant l'ensemble des données.

4 Visualisation et Analyse des Résultats :

Après la mise en place du module de détection des anomalies (Detection_anomalies.py), les résultats sont indexés dans Elasticsearch lorsqu'un patient présente des valeurs anormales.
Kibana permet ensuite de visualiser ces données sous forme de tableaux de bord interactifs afin d'analyser le comportement du système et la nature des anomalies détectées.
Le tableau de bord réalisé comprend quatre visualisations principales.

Description des visualisations réalisées :

Nombre total de cas anormaux :

La première visualisation affiche le nombre total d'enregistrements correspondant à des anomalies détectées.
Ce compteur global représente le total des cas où la fonction detecter_anomalies () a renvoyé une liste non vide. Chaque valeur comptabilisée correspond à un patient dont la pression systolique ou diastolique dépasse les seuils médicaux définis :

- systolique > 140 mmHg : hypertension_systolique
- systolique < 90 mmHg : hypotension_systolique
- diastolique > 90 mmHg : hypertension_diastolique
- diastolique < 60 mmHg : hypotension_diastolique

Évolution temporelle

La deuxième visualisation représente l'évolution des anomalies dans le temps sous forme de graphique empilé.
Chaque barre correspond à une période donnée, et les différentes couleurs représentent les types d'anomalies détectées.

Cette visualisation permet d'observer :

- la stabilité ou la variation du nombre d'anomalies
- la proportion relative des différents types d'anomalies
- la répartition entre hypertension et hypotension

Dans notre cas, les variations observées sont principalement dues à la génération aléatoire des données, ce qui explique l'absence de tendance médicale réelle.

Distribution des types d'anomalies

Un graphique en secteurs (camembert) montre la répartition des anomalies détectées. Chaque portion correspond à un type d'anomalie généré par la fonction detecter_anomalies () :

- hypertension_systolique
- hypotension_systolique
- hypertension_diastolique
- hypotension_diastolique

Cette visualisation permet d'identifier rapidement quel type d'anomalie est le plus fréquent dans le système.

On observe généralement une proportion importante d'hypertension, ce qui s'explique par les plages larges utilisées lors de la génération des valeurs (70-200 mmHg pour la systolique et 40-130 mmHg pour la diastolique).

Identification des cas critiques

La dernière visualisation est un tableau détaillé des patients présentant des anomalies.

Ce tableau affiche :

- l'identifiant du patient
- son nom
- la valeur systolique
- la valeur diastolique
- le type d'anomalie détectée

Cette visualisation est importante car elle permet d'identifier les cas les plus critiques.

Dans un contexte médical réel, ces cas pourraient déclencher une alerte immédiate.

Analyse des résultats obtenus

L'analyse du tableau de bord confirme que le module detecter_anomalies() fonctionne correctement.

Les patients affichés dans le tableau des cas critiques présentent bien des valeurs dépassant les seuils médicaux définis dans le code. Cela montre que les règles de comparaison implémentées dans la fonction sont cohérentes et appliquées correctement.
La distribution des anomalies montre une répartition relativement équilibrée entre les différents types, ce qui est cohérent avec la génération aléatoire des données.
L'évolution temporelle ne révèle pas de tendance spécifique, ce qui est logique car les données sont simulées et ne proviennent pas d'un contexte clinique réel.

Globalement, le tableau de bord Kibana valide le bon fonctionnement du pipeline complet :

Génération des données puis transmission via Kafka puis détection des anomalies puis indexation Elasticsearch et enfin visualisation Kibana.

Ce résultat démontre la cohérence et l'intégration réussie de l'architecture Big Data mise en place.

5 Discussion :

Limites des seuils

Les seuils utilisés sont fixes et identiques pour tous les patients. En réalité, les seuils peuvent varier selon l'âge, le sexe, les antécédents médicaux ou le contexte clinique.

Par exemple, une valeur de 140 mmHg peut être tolérée dans certains cas mais critique dans d'autres. Le système actuel ne tient pas compte de ces variations individuelles.

Limites du modèle

Le système repose uniquement sur des règles simples basées sur des seuils. Il ne s'agit pas d'un modèle prédictif ou intelligent.

Il ne détecte pas :
- les tendances progressives d'augmentation de la pression
- les variations anormales sur plusieurs mesures successives
- les corrélations avec d'autres paramètres médicaux

De plus, les données sont générées aléatoirement et ne reflètent pas un contexte clinique réel.

Améliorations possibles

Plusieurs améliorations pourraient être envisagées :

Intégrer un modèle de Machine Learning pour détecter des anomalies plus complexes. Analyser les tendances temporelles sur plusieurs mesures d'un même patient. Ajouter d'autres paramètres médicaux comme le rythme cardiaque ou la saturation en oxygène. Mettre en place un système d'alerte automatique en cas de valeurs critiques.

Ces améliorations permettraient de rendre le système plus proche d'une application hospitalière réelle.

Sécurité des données médicales

Dans un environnement réel, les données médicales sont sensibles et doivent être protégées.

Il serait nécessaire de :

- sécuriser les communications entre Kafka, Elasticsearch et Kibana
- activer l'authentification et le chiffrement TLS

- anonymiser les données des patients
-respecter les réglementations comme le RGPD

Dans ce projet, les données sont simulées, ce qui évite les problématiques de confidentialité. Cependant, la sécurité reste un aspect essentiel dans toute application médicale réelle.

Conclusion :

Ce projet nous a permis de mettre en place un système complet de surveillance de la pression artérielle en temps réel. Nous avons réussi à créer une chaîne de traitement allant de la génération des données médicales jusqu'à leur affichage dans Kibana. Kafka permet de transmettre les données en continu, Elasticsearch stocke les cas anormaux, et Kibana permet de les visualiser facilement grâce à des tableaux de bord clairs.

Même si les données sont simulées et que la détection repose uniquement sur des seuils simples, le système fonctionne correctement et montre comment une architecture Big Data peut être utilisée dans un contexte médical. Ce projet constitue donc une bonne base pour aller plus loin, par exemple en ajoutant un modèle plus intelligent ou en renforçant la sécurité des données dans un environnement réel.